LÖSUNGEN · LEHREREXEMPLAR

W4·3 – Umformen und Anwendungen: Lösungen

Zum Arbeitsblatt `AQ_W4_AB3_Umformen_und_Anwendungen.docx` · IDE: `AQ_W4_L4_Hafen_LSG.txt` (Umformen, Expertenpfad, Knoten, MeinStapel) · alle Zustände, Traces, Rückgaben, Markier-Reihenfolgen und Verschiebungszahlen mit Java (javac 21) und exakten Nachbauten von Stack, Queue und DynArray nachgerechnet; Klammerprüfung, Postfix und fluteIterativ sind die Musterlösungen von Level 4.5/4.6 · Stapel oben zuerst, Schlange vorne zuerst, Reihung Index 0 zuerst, – = leer

Teil 1 (DS 28)

Aufgabe 1 – Umladen im Hafen (7 BE)

nach Zeile	d	s	q	e
4	Holz, Erz, Gold, Salz	Salz, Gold, Erz, Holz	–	–
8	Holz, Erz, Gold, Salz	–	Salz, Gold, Erz, Holz	–
12	Holz, Erz, Gold, Salz	–	–	Holz, Erz, Gold, Salz

a) 3 BE; d bleibt unverändert, weil getItem nur liest. **b)** e hat **dieselbe Reihenfolge** wie d und enthält **dieselben Objekte** (mit Java geprüft: true) – umgeladen werden nur Referenzen, keine Kisten kopiert (1 BE). **c)** Mit append: e = **Salz, Gold, Erz, Holz** (1 BE).

d) d → s (Zeilen 2–4): Index 0 liegt unten, gelesen von oben ist die Reihenfolge **umgedreht**. s → q (Zeilen 6–8): das oberste Element wird vorne angestellt – die (umgedrehte) Reihenfolge bleibt. q → e (Zeilen 10–12): `insertAt(0, …)` setzt jedes neue Element nach vorne – **zweite Umkehrung**. Zweimal umgedreht ergibt die ursprüngliche Reihenfolge (2 BE).

Aufgabe 2 – Klammern im Frachtbrief (9 BE)

Zeichen	Aktion	Stapel (oben → unten)
[	push	[
(	push	(, [
)	pop → (passt	[
{	push	{, [
}	pop → { passt	[
]	pop → [passt	–
Ende	Stapel leer → true	–

a) 3 BE; Buchstaben, Ziffern und Rechenzeichen werden ignoriert. Rückgabe **wahr**.

Ausdruck	Ergebnis	Begründung (Positionen ab 0 gezählt)
`([]{)}`	falsch	Bei `)` (Position 4) liegt `{` oben – die Art passt nicht.
`((()))`	falsch	Die letzte `)` (Position 4) kommt bei **leerem** Stapel – ohne Leer-Abfrage Laufzeitfehler.
`{[}`	falsch	Bei `}` liegt `[` oben – `[` wurde nie geschlossen.
`x)(`	falsch	`)` (Position 1) bei leerem Stapel; danach bliebe `(` offen.

b) je 1 BE (mit Java geprüft). **c)** `([)]` und `)(` liefern beim Zählen 0 und würden als richtig gelten (Java: true bzw. true), sind aber falsch. Der Zähler kennt weder die **Art** der offenen Klammer noch ihre **Reihenfolge**; eine schließende Klammer muss zur **zuletzt** geöffneten passen – das kann nur ein Stapel (LIFO) liefern (2 BE).

Aufgabe 3 – Der Rechenschieber des Zollmeisters (9 BE)

Zeichen	Aktion	Stapel (oben → unten)
7	push 7	7
4	push 4	4, 7
2	push 2	2, 4, 7
-	b = 2, a = 4, push 4 − 2 = 2	2, 7
*	b = 2, a = 7, push 7 · 2 = 14	14
3	push 3	3, 14
+	b = 3, a = 14, push 14 + 3 = 17	17
Ende	pop → 17	17

a) 3 BE; Ergebnis **17** = 7 · (4 − 2) + 3. **b)** (1) (5 + 3) · (9 − 7) → `53+97-*` = **16** · (2) 8 − 2 − 3 → `82-3-` = **3** · (3) 8 − (2 − 3) → `823--` = **9** (3 BE; alle mit dem Postfix-Rechner der Musterlösung geprüft). Gleichwertig zu (1): `53+97-*` ist die einzige Lösung mit den Operanden in dieser Reihenfolge; `97-53+*` (Faktoren vertauscht) liefert ebenfalls 16.

c) Ergebnis **-11**: Statt 4 − 2 wird 2 − 4 = −2 gerechnet, 7 · (−2) = −14, −14 + 3 = −11. Betroffen sind nur Rechenzeichen, bei denen die Reihenfolge der Operanden zählt (hier „−"); + und · sind kommutativ (2 BE). **d)** Höchstens **4** Werte (2, 5, 3, 4). Beispiel mit höchstens zwei: `12+3+4+5+` (Ergebnis 15) – nach jeder neuen Ziffer folgt sofort ein Rechenzeichen (1 BE).

Aufgabe 4 – Level 4.5

Musterlösungen und häufige Fehler: `AQ_W4_L4_Hafen_LSG.txt`, Klasse Umformen (7 Tests). Kontrollwerte aus dem Hauptprogramm: `klammernOk("{[(a+b)*c]}")` = true · `klammernOk("([)]")` = false · `postfix("34+2*")` = 14 · nachGewicht(kai, 5) = Holz, Erz, Salz. schlangeZuDynArray(Standard-Schlange) = Holz, Erz, Gold, Salz, Tuch, Wein (q danach unverändert), dynArrayZuStapel(Standard-Ladung) von oben: Wein, Tuch, Salz, Gold, Erz, Holz.

Teil 2 (DS 29, Kür)

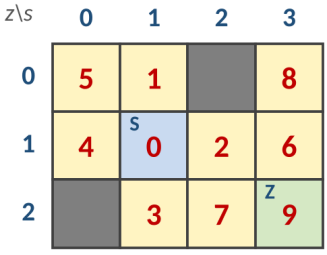
Aufgabe 5 – Die Flut auf der Lichtung (13 BE)

a) Schlange (4 BE):

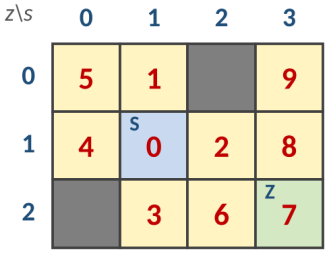
Nr.	entnommen	neu markiert	Schlange (vorne → hinten)
0	–	(1|1)	(1|1)
1	(1|1)	(0|1) (1|2) (2|1) (1|0)	(0|1) (1|2) (2|1) (1|0)
2	(0|1)	(0|0)	(1|2) (2|1) (1|0) (0|0)
3	(1|2)	(1|3) (2|2)	(2|1) (1|0) (0|0) (1|3) (2|2)
4	(2|1)	–	(1|0) (0|0) (1|3) (2|2)
5	(1|0)	–	(0|0) (1|3) (2|2)
6	(0|0)	–	(1|3) (2|2)
7	(1|3)	(0|3) (2|3)	(2|2) (0|3) (2|3)
8	(2|2)	–	(0|3) (2|3)
9	(0|3)	–	(2|3)
10	(2|3)	–	(leer)

b) Stapel (4 BE):

Nr.	entnommen	neu markiert	Stapel (oben → unten)
0	–	(1|1)	(1|1)
1	(1|1)	(0|1) (1|2) (2|1) (1|0)	(1|0) (2|1) (1|2) (0|1)
2	(1|0)	(0|0)	(0|0) (2|1) (1|2) (0|1)
3	(0|0)	–	(2|1) (1|2) (0|1)
4	(2|1)	(2|2)	(2|2) (1|2) (0|1)
5	(2|2)	(2|3)	(2|3) (1|2) (0|1)
6	(2|3)	(1|3)	(1|3) (1|2) (0|1)
7	(1|3)	(0|3)	(0|3) (1|2) (0|1)
8	(0|3)	–	(1|2) (0|1)
9	(1|2)	–	(0|1)
10	(0|1)	–	(leer)

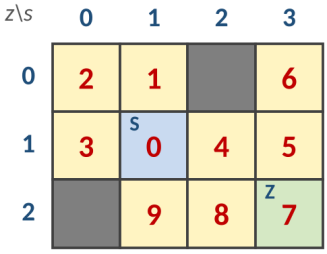


Schlange: Markier-Reihenfolge

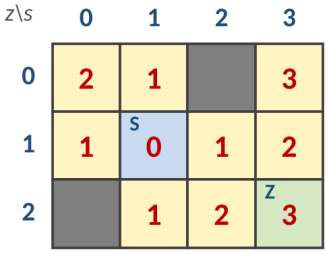


Stapel: Markier-Reihenfolge

c) Beide Aufrufe liefern **8** und hinterlassen dieselbe Karte `++#+ / +S++ / #++Z` – gelüftet wird immer genau die Menge der von S aus erreichbaren Felder. Verschieden ist die **Reihenfolge** (spur, siehe Karten). Die **Schlange** arbeitet in Wellen: erst alle Felder mit Abstand 1 von S, dann Abstand 2, dann 3 (Breitensuche). Größte Füllung der Ablage: Schlange 5, Stapel 4 Positionen (2 BE).



rekursiv (Welt 3)



Wellen: Abstand zu S

d) Die Stapel-Variante markiert **alle** freien Nachbarn eines Feldes sofort beim Einfügen – z. B. werden von (1|1) aus (0|1), (1|2), (2|1) und (1|0) auf einmal markiert. Erst danach geht sie vom zuletzt eingefügten Feld (1|0) aus in die Tiefe. Die Rekursion markiert dagegen ein Feld erst, wenn sie es **betritt**, und geht sofort vom ersten Nachbarn (0|1) weiter in die Tiefe; die anderen Nachbarn kommen erst nach der Rückkehr dran. Mit „Markieren beim Entnehmen“ und umgekehrter Nachbarreihenfolge käme man der Rekursion näher (2 BE).

e) Aus pos müssen z und s eindeutig zurückgewonnen werden: z = pos / 1000, s = pos % 1000 klappt für alle Karten mit weniger als 1000 Spalten (z. B. (2|3) → 2003 -> 2|3). Mit z · 10 + s fielen ab 10 Spalten Positionen zusammen, z. B. (1|12) → 22 = (2|2) (1 BE).

Aufgabe 6 – Eine Frage des Aufwands (6 BE)

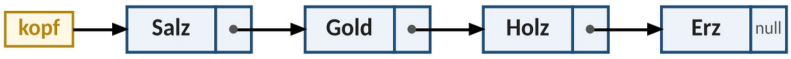
n	5	10	100	1000
Verschiebungen beim Entnehmen (delete(0))	10	45	4950	499500
Verschiebungen beim Einfügen vorne (insertAt(0, k))	10	45	4950	499500
n · (n − 1) / 2	10	45	4950	499500

a) Beim ersten delete(0) rücken n − 1 Elemente nach, beim zweiten n − 2 usw.: (n − 1) + (n − 2) + … + 1 + 0 = **n · (n − 1) / 2**, für n = 5 also 4 + 3 + 2 + 1 = 10 (Zählung mit einer instrumentierten Nachbildung; append verschiebt nichts). Der Aufwand wächst **quadratisch**, mit einer Schlange nur **linear** (3 BE). **b)** Ebenfalls 0 + 1 + … + (n − 1) = n · (n − 1) / 2 Verschiebungen (1 BE).

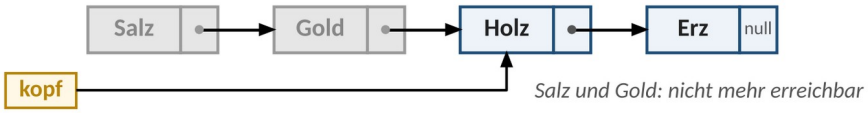
c) Dynamische Reihung, wenn man auf **beliebige Positionen** zugreifen muss: sortiert einfügen, aus der Mitte entfernen, über den Index suchen oder ersetzen (Ladeliste, Buchungsliste). Schlange, wenn nur nach FIFO gearbeitet wird und viele Elemente durchlaufen – jede Operation hat konstanten Aufwand (Warteschlangen, Flood-Fill) (2 BE). Hinweis: Wie DynArray in der Online-IDE intern gebaut ist, legt die Schnittstelle nicht fest – die Rechnung gilt für eine Implementierung mit Reihung.

Aufgabe 7 – Zeigerbilder (6 BE)

a) kopf → Salz → Gold → Holz → Erz → null (1 BE):



b) Rückgaben **Salz, Gold**; danach kopf → Holz → Erz → null. Salz und Gold sind nicht mehr erreichbar (2 BE):



c) Ergebnis kopf → Salz → null: Der neue Knoten zeigt auf null statt auf den alten kopf – Gold, Holz und Erz sind abgehängt und verloren, der Stapel enthält nur noch Salz (2 BE):



d) top() und pop() lesen `kopf.getInhalt()`; bei leerem Stapel ist kopf null → NullPointerException statt der geforderten IllegalStateException (Java: leerer MeinStapel → IllegalStateException durch die Leer-Abfrage). push braucht keine Abfrage: Bei leerem Stapel wird null einfach zum Nachfolger des neuen Knotens (1 BE).

Aufgabe 8 – Level 4.6

Musterlösung und häufige Fehler: `AQ_W4_L4_Hafen_LSG.txt`, Klassen Expertenpfad, Knoten, MeinStapel (12 Tests, davon 4 im Block LEVEL-4.6). Kontrollwerte: Höhle ab (1|1) 12 Felder (Schlange und Stapel), Lichtung 8 Felder; spur der Lichtung wie in Aufgabe 5 (Schlange: (1,1) (0,1) (1,2) (2,1) (1,0) (0,0) (1,3) (2,2) (0,3) (2,3); Stapel: (1,1) (0,1) (1,2) (2,1) (1,0) (0,0) (2,2) (2,3) (1,3) (0,3)). Hauptprogramm: „MeinStapel abgebaut: Gold Holz Erz“.

Für Schnelle

Schlange aus zwei Stapeln: Ausgabe A, B, C, D – die richtige FIFO-Reihenfolge, weil das Umschichten von ein nach aus die Reihenfolge genau einmal umdreht. Insgesamt 4 Umschichtungen (A, B, C beim ersten dequeue, D beim vierten); jedes Element wird höchstens einmal umgeschichtet. **Wellen zählen:** siehe Karte „Abstand zu S“; die Schlange erreicht jedes Feld zuerst über einen Weg mit möglichst wenigen Schritten (Breitensuche) – anders als das Backtracking aus Welt 3, das irgendeinen Weg findet. **MeinSchlange:** Beim ersten enqueue müssen kopf **und** ende auf den neuen Knoten zeigen; wird der letzte Knoten entnommen, muss auch ende wieder null werden.